

Human-Machine Dialogue Project Report

Alberto Messa – 258855
Dept. of Information Engineering and Computer Science
University of Trento, Italy
alberto.messa@studenti.unitn.it
GitHub Repository Link

Abstract—This project presents the implementation and evaluation of a task oriented chatbot for the customer service of an aquatic center. The system is designed to support common front desk tasks, such as requesting information, booking services, modifying or cancelling existing bookings, reporting lost items and managing a simple item shop.

The chatbot is designed as a modular hybrid architecture that combines LLM-based components with deterministic logic. LLMs are used for language understanding and response generation, while deterministic modules handle dialogue state tracking, value validation and database queries. This approach allows the system to provide natural responses and reliability at the same time.

I. INTRODUCTION

A. Dialogue System Description

1) *Concept and Motivation*: More and more services now use autonomous systems to handle either the initial state or the entire interaction with customers. Examples can be found from ticket machines and self-ordering kiosks to online customer support bots. In this project, the aim is to design a chatbot for the customer service of an aquatic center to help users in realistic scenarios usually handled by front desk operator.

2) *Handled Exchanges, Inputs, and Outputs*: The chatbot supports three main categories of exchanges. **Informative requests**, where the user asks questions about opening hours, prices, available services or facility rules. **Transactional requests**, where the user wants to perform an action, such as booking a course or a spa session, modifying an existing booking, or cancelling a reservation. **Secondary requests**, such as reporting lost items and interacting with the shop of the aquatic center.

The system can process both complete and partial input sentences. For example, users may write full requests such as “I want to book a spa session for tomorrow at 6 PM”, or shorter contextual answers such as “Tomorrow”, “At 6 PM”, or “Yes, I confirm”, and moreover, the chatbot can also process messages containing multiple requests.

Then, depending on the dialogue state, it can provide information, ask for missing details, suggest valid options, request confirmation and recover a previously paused request.

B. User Group

The system is designed for users who interact with the aquatic center, so users don’t need any specific domain knowledge or technical terminology. The variety of intents handled by the bot should cover the majority of users’ interactions with an aquatic center.

II. CONVERSATION DESIGN

A. Types of Dialogues

The system is designed to support task-oriented dialogues, where the goal is to complete a deterministic schema over the conversation. Some interactions are single-turn, for example when the user asks for precise information, while others are multi-turn and require a process of collecting missing information in order to complete the task.

The chatbot also supports more complex cases, such as multi-intent utterances and task recovery, that are handled through strategies described in the following sections.

B. Interaction Properties

1) *Initiative and Engagement*: The dialogue follows a mixed initiative approach. At the beginning of the interaction, the user is free to introduce a request, provide details, ask questions or combine multiple intents in the same utterance. Once the system identifies the user’s goal, the interaction becomes more guided and the chatbot asks targeted questions to collect the missing information needed to complete the task.

Engagement is maintained by keeping the conversation short and direct on the current goal. The system avoids asking again for information that has already been provided and uses the dialogue context to interpret short answers, corrections, confirmations and choices among previously offered options.

2) *User Profiles*: The system is designed to handle different user profiles. **Under-informative users** provide part of the required information, so the chatbot guides them step by step by asking for the missing slots. In these cases, the system can follow the expected task structure.

Over-informative users may provide many details at once or combine multiple requests in the same utterance. The system tries to extract only the relevant information and ignore unnecessary details. However, a limitation can arise when the user message contains a long introduction or unrelated information before the actual request, because the system processes *at most the first two main intents in their order of appearance*.

The chatbot can also handle different levels of coherence. A **low-coherence user** may change previously provided values, introduce contradictory information or move to another topic. In these cases, the system updates the dialogue state and continues the interaction instead of blocking the conversation.

3) *Acknowledgement and Confirmation*: The system uses acknowledgements to keep the conversation natural and to signal that the user’s input has been understood. However, it does not explicitly acknowledge every slot change, because this would make the dialogue repetitive. Instead, the chatbot usually includes part of the collected information in its responses, allowing the user to notice possible errors or unwanted values.

Full confirmations are used for important actions such as bookings, modifications and cancellations. In these cases, the chatbot summarizes the task details and asks the user to confirm before completing the operation.

4) *Conversational Markers*: Conversational markers are used to make the interaction smoother and more understandable. For example, when the system handles multiple requests, it can introduce a step-by-step response to make clear that only one request will be processed. When a queued task is recovered, the chatbot can use a transition to return naturally to the previous topic.

5) *Fallback Responses and Error Recovery*: Fallback responses are used when the system cannot recognize any useful intent and classifies the intent as out-of-scope. In this case, the chatbot briefly explains what kinds of requests it can handle.

Error recovery is mainly related to the precise understanding and slot validation. If a provided value cannot be mapped to any valid option during domain validation, the system returns an error for that specific slot. In this case, the chatbot asks the user to repeat or reformulate only the problematic value, without blocking the conversation or restarting the whole task.

III. CONVERSATION MODEL AND ARCHITECTURE

A. Pipeline Overview

From the architectural point of view, the system is implemented as a modular pipeline that combines LLM-based components with deterministic control logic. The hybrid approach exploits the linguistic flexibility of LLMs, while keeping the most critical control operations under deterministic logic. The system is divided into multiple phases: utterance segmentation and intent routing, slot extraction, dialogue state tracker, database validation, dialogue management and natural language generation.

The complete pipeline diagram is reported in Appendix A.

B. Router and Natural Language Understanding

The **Router** analyzes the latest user utterance, splits it into meaningful segments and assigns an intent to each segment. This last step is extremely useful because it allows the **NLU** component to focus only on slot extraction, since user’s intent is already known.

Before the router call, the conversation history can be filtered to remove segments that belong to the queue. This helps the Router to focus on the current user intent, without being misled by old segments associated to already paused requests.

Moreover, a post-processing is applied to the router output in the following cases. Since many tasks require user’s name

and surname to be completed, if a short messages contains only this information it can be ambiguous for the LLM understand the correct intent. So, these segments are first classified as `user_identification` and then merged into the previous active intent, if present. Similarly, if model returns more than two segments, the output is truncated deterministically to keep the interaction controlled.

After routing, the NLU receives the target segment, the target intent, the full user message and part of the conversation history. It extracts only the slots from the target segment, using intent-specific prompts that help to avoid ambiguity on processing unnecessary schemas.

C. Dialogue State Tracking and Database Validation

The NLU output is then processed by the **Dialogue State Tracker**. The DST has the role of maintaining the active intent and its collected slots. Moreover, it handles context switch between intents, fills missing slot keys in intent schema and store name and surname when they’re provided. It uses important functions to normalize and validate all the extracted data, such as string and temporal expressions.

Moreover, the DST prevents unsafe confirmations. For example, when the user confirms a task while changing some slot values in the same turn, the confirmation is set to null. This ensure that important actions are confirmed only after all other slots have been correctly processed in previous turns.

After the dialogue state is updated, the **database layer** validates the current request using mock domain knowledge. In this project, the database directly determines the next best action to execute. For this reason it’s queried at each turn.

For each intent the DB layer follows a specific sequence of action to decide whether the system should ask for a missing slot, validate a new slot, report an invalid value or complete the task. The DB knows when a task has been completed and provides information needed to answer the user.

This design choice avoids relying on LLM to predict specific domain outcomes. Instead, using deterministic logic makes the system more reliable and easier to control.

D. Dialogue Manager and Natural Language Generation

The **Dialogue Manager** receives the database result and simply converts it into a structured next best action. Since all domain validation are already done by the database layer, the DM mainly organizes the output into a controlled JSON response. This structure contains the NBA and other information such as the slot to request, available suggestions or blacklist values provided by the mock DB.

The next best action covers different scenarios, such as asking for a missing slot, reporting an invalid value or asking for clarification, providing requested information and confirming task completion.

The **Natural Language Generator** receives the next best action, the dialogue state and the recent conversation history and finally produces the user response in natural language. Like the NLU, NLG uses intent-specific prompts. Moreover, the code injects some rules at the end of the prompt when

special flags are active, such as step-by-step handling, multi-task responses, second responses, or queue recovery.

E. Prompt Design

The prompts used in the system follow a common structure across the main LLM-based components. Each prompt is organized around four main elements:

- **Component role:** defines the role of the model, such as Router, NLU, Dialogue Manager, or Natural Language Generator.
- **Input description:** explains the input structure and how each field should be interpreted.
- **Rules and output format:** specifies the constraints that the model must follow. For structured components, the output must be raw valid JSON, without Markdown blocks, explanations, or additional text.
- **Examples:** provides input-output examples for common or ambiguous cases, such as short answers, multi-intent messages, invalid values, confirmations, or slot corrections.

As already mentioned for the NLU and NLG components, some prompts are dynamically specialized according to the target intent. These intent-specific prompts provide additional context, specific rules, and examples related only to the current intent. This avoids giving the model unnecessary information that could introduce ambiguity, especially in cases where different intents have similar slots, such as modification and cancellation requests.

This structure makes the prompts explicit and helps the LLMs produce outputs that are consistent with the pipeline requirements.

F. Intents and Slots Definition

The supported intents and slots are reported in Appendix B. The same intent-slot schema is used by the NLU component for slot extraction and by the Dialogue State Tracker to maintain the active dialogue state.

Slots marked as categorical are expected to be mapped to a predefined set of values. The NLU is instructed to associate the user’s wording with one of these values when the meaning is clear, while the Dialogue State Tracker normalizes and validates the extracted value before updating the dialogue state. This helps reduce variability in the model output and makes the following database validation more reliable.

IV. EVALUATION

A. Evaluation Setup

The evaluation is designed to test the four LLM-based components of the pipeline: Router, NLU, DM and NLG. Each component is evaluated separately in order to analyze module weaknesses

The comparison is performed on the models listed in Appendix C. Phi-4-mini is excluded from the final comparison because it doesn’t reliably output the JSON responses required by system. In fact, it often generates additional text after the expected JSON output.

B. Data Description and Generation

The ground truth dataset was synthetically generated with GPT-5.5 Thinking, using the project domain, component prompts, intent schemas, database constraints and expected pipeline behavior as context. The goal was to create controlled test cases covering common and ambiguous situations such as: short contextual answers, multi-intent messages, missing slots, invalid values, confirmations, slot corrections and out-of-scope inputs.

The Router dataset contains 200 samples: about 140 single segment utterances and 60 multi-segment utterances with two or three intents. The NLU dataset also contains 200 samples, with roughly 15 examples for each intent. The DM dataset contains 240 samples and covers all main database validation cases. And the NLG dataset contains 34 samples, corresponding to 37 evaluated DM actions because of multi-response cases.

Using an LLM to generate the evaluation data was similar to manually writing test cases for the project. The generated samples were checked to ensure that they followed the expected prompt behavior and respected the database constraints. However, components such as the Router and the NLU cannot be tested on every possible user input. Their size and variety still provide a useful estimate of how the models will perform in the most important scenarios.

C. Intrinsic Evaluation

The intrinsic evaluation is performed by testing each LLM-based component against its ground truth. Since each component has different goal and output structure, the evaluation does not rely on a single metric such as a global F1 score. Instead, each component is evaluated with the metric that best reflects whether its output can be correctly used by the next component in the pipeline.

- **Router – Segment accuracy:** a prediction is considered correct only if the number of segments is correct and each segment has the correct intent.
- **NLU – Slot F1:** True positives are correctly predicted non-null slots, false negatives are expected slots that were missed and false positives are extra or wrongly predicted slots.
- **Dialogue Manager – Deterministic accuracy:** verifies whether the predicted JSON exactly matches the expected next best action, including slot, options, blacklist, and enriched data.
- **NLG – Constraint-based acceptance rate:** evaluates whether the generated response satisfies automatic constraints, such as question form, required information, valid options, response length and absence of false promises. Responses that satisfy all constraints are accepted automatically, while failures were manually inspected before computing the final NLG score.

D. Extrinsic Evaluation

The extrinsic evaluation is performed after the automatic component evaluation, using the complete chatbot through the

UI notebook prepared for testers.¹ Testers are free to interact with the system, choosing the tasks they consider most relevant or interesting.

After the interaction, testers complete an anonymous Google Form. The form uses a 1–5 Likert scale, from strongly disagree to strongly agree and evaluates the chatbot along five main categories: **request understanding, task success and efficiency, context tracking, response naturalness, and robustness to unclear or invalid inputs**. An open question is also included to collect qualitative feedback about what did not work well or what should be improved.

E. Intrinsic Evaluation Results

The intrinsic evaluation shows that the tested models behave differently across the four LLM-based components. Gemma3-4B obtains the best Router score, with an accuracy of 0.955, while Qwen3-4B achieves the strongest overall performance. In particular, Qwen3-4B obtains the best NLU score, with a slot F1 of 0.936, reaches perfect scores on the Dialogue Manager and NLG evaluations.

The most evident weakness is the low NLU score obtained by Mistral-7B, which reaches only 0.529. This result is likely influenced by output-formatting and schema-following issues, since many predictions contained empty slot dictionaries. In this pipeline, following the required JSON schema is as important as understanding the user input.

For this reason, Qwen3-4B was selected as the model to be used in the following extrinsic evaluation with external testers. The complete intrinsic results are reported in Appendix D.

F. Extrinsic Evaluation Results

The extrinsic evaluation was conducted with 5 external testers. Although the number of responses is limited, the results provide a preliminary indication of the behavior of the complete chatbot. Overall, the system obtained an average score of 3.68 out of 5, with a standard deviation of 0.28 across testers.

The strongest result is obtained for response naturalness, with an average score of 4.40. Task success and efficiency also obtain a positive result, with an average score of 4.00. Request understanding reaches 3.60, suggesting that the chatbot generally understood the users’ goals. The weakest dimensions are context tracking and robustness to unclear or invalid inputs, both with an average score of 3.20. This suggests that the main limitations of the complete system are related to maintaining context across complex interactions and recovering from problematic user inputs.

The complete extrinsic evaluation results are reported in Appendix E.

G. Error Analysis and Limitations

The intrinsic evaluation shows that Qwen3-4B is the strongest model overall. However, this result should also be interpreted considering the development process: most prompt

refinements and intermediate observations were made while testing Qwen3-4B. This may have made the final pipeline more aligned with its behavior than with the behavior of the other models.

The results obtained with Mistral-7B and the preliminary tests with Phi-4-mini also show an important limitation of LLM-based pipelines. Even when the same task and the same evaluation data are used, models may react very differently to the same prompt structure. In particular, strict JSON generation and schema following were not equally reliable across models. Small changes in the prompt could also produce different results on the same test cases, showing that prompt-based components can be powerful but also difficult to control completely.

The extrinsic evaluation is mainly limited by the small number of testers. Since only 5 responses were collected, the results should be interpreted as preliminary feedback rather than as a statistically strong evaluation. The results are still useful to identify the most visible strengths and weaknesses of the complete chatbot.

The score obtained for context tracking can be considered acceptable, because maintaining dialogue context is one of the most difficult aspects of an interactive agent. Users naturally expect the system to remember information mentioned several turns earlier, while an agent requires explicit and rigorous logic to track even a limited amount of dialogue history correctly.

The score on unclear, incomplete or invalid inputs should be interpreted with caution. The mostly neutral feedback may simply mean that testers did not try many problematic cases. Therefore, this result does not necessarily indicate a major weakness of the system. Invalid values are also handled through a controlled pipeline, combining NLU extraction, DST normalization and deterministic database validation.

V. CONCLUSION

This project showed that a modular hybrid architecture can be effective for building a task-oriented chatbot in a realistic service domain. By combining LLM-based components with deterministic dialogue tracking, database validation, and control logic, the system was able to handle different types of user requests while keeping the interaction reasonably structured and reliable.

An interesting result is that relatively small open-weight models, such as 3B–7B parameter models, can already provide useful performance when the task is well structured and the prompts are carefully designed. This suggests that even stronger results could be obtained with larger and more capable models, especially for context tracking and more natural multi-turn interactions.

¹https://github.com/MessaAlberto/HMD_aquatic_center_chatbot/blob/main/notebooks/UI_chatbot.ipynb

APPENDIX A PIPELINE OVERVIEW

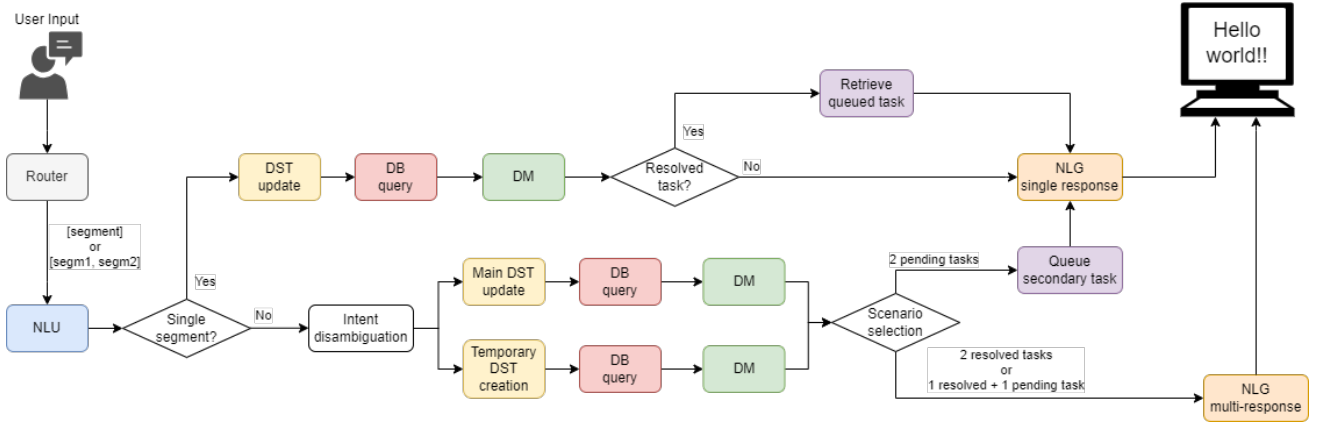


Fig. 1. Overview of the modular pipeline used by the dialogue system.

APPENDIX B LIST OF INTENTS AND SLOTS

TABLE I: Detailed intent-slot schema used by the chatbot.

Intent	Slot	Type	Description
ask_opening_hours	facility_type	categorical	Facility whose opening hours are requested.
	date	temporal	Date or day referred to by the user.
	time	temporal	Time or time-of-day expression.
ask_pricing	service_type	categorical	Service or facility for which the price is requested.
	sub_type	categorical	Type of pass or subscription.
	user_category	categorical	User category used for price selection.
ask_rules	topic	categorical	Facility or area related to the rule.
	specific_inquiry	text	Specific rule, object, or behavior mentioned by the user.
book_course	course_activity	categorical	Course the user wants to book.
	target_age	categorical	Age group for the course.
	level	categorical	Course level requested by the user.
	day_preference	categorical	Preferred day for the course.
	name	text	User's first name.
	surname	text	User's last name.
	confirmation	categorical	Final user confirmation or denial.
book_spa	date	temporal	Date of the spa booking.
	time	temporal	Time of the spa booking.
	people_count	integer	Number of people for the booking.
	name	text	User's first name.
	surname	text	User's last name.
	confirmation	categorical	Final user confirmation or denial.
modify_booked_course	name	text	User's first name.
	surname	text	User's last name.
	course_activity_old	categorical	Course activity in the existing booking.
	target_age_old	categorical	Age group in the existing booking.
	level_old	categorical	Level in the existing booking.
	day_preference_old	categorical	Day in the existing booking.
	course_activity_new	categorical	New requested course activity.
	target_age_new	categorical	New requested age group.
	level_new	categorical	New requested course level.
	day_preference_new	categorical	New requested course day.
	confirmation	categorical	Final user confirmation or denial.
modify_booked_spa	name	text	User's first name.
	surname	text	User's last name.
	date_old	temporal	Date of the existing spa booking.
	time_old	temporal	Time of the existing spa booking.

Intent	Slot	Type	Description
	people_count_old	integer	Number of people in the existing booking.
	date_new	temporal	New requested date.
	time_new	temporal	New requested time.
	people_count_new	integer	New requested number of people.
	confirmation	categorical	Final user confirmation or denial.
cancel_booked_course	course_activity	categorical	Course activity to cancel.
	target_age	categorical	Age group of the booked course.
	level	categorical	Level of the booked course.
	day_preference	categorical	Day of the booked course.
	name	text	User's first name.
	surname	text	User's last name.
cancel_booked_spa	confirmation	categorical	Final user confirmation or denial.
	date	temporal	Date of the spa booking to cancel.
	time	temporal	Time of the spa booking to cancel.
	people_count	integer	Number of people in the booking.
	name	text	User's first name.
buy_equipment	surname	text	User's last name.
	confirmation	categorical	Final user confirmation or denial.
	item	categorical	Shop item requested by the user.
	size	categorical	Requested item size.
	color	categorical	Requested item color.
report_lost_item	brand	categorical	Requested item brand.
	confirmation	categorical	Final user confirmation or denial.
	lost_item	text	Object lost by the user.
	item_color	text	Color of the lost item.
	last_seen_location	text	Place where the item was last seen.
user_identification	last_seen_date	temporal	Date when the item was last seen.
	name	text	User's first name.
	surname	text	User's last name.
	name	text	User's first name.
	surname	text	User's last name.
greeting_closing	–	–	No slot extraction required.
out_of_scope	–	–	No slot extraction required.

APPENDIX C EVALUATED MODELS

TABLE II
MODELS USED IN THE AUTOMATIC COMPONENT EVALUATION.

Model Name	Model Identifier
Qwen3-4B	Qwen/Qwen3-4B-Instruct-2507
Qwen2.5-3B	Qwen/Qwen2.5-3B-Instruct
Gemma3-4B	google/gemma-3-4b-it
Mistral-7B	mistralai/Mistral-7B-Instruct-v0.3

APPENDIX D INTRINSIC EVALUATION RESULTS

TABLE III
INTRINSIC EVALUATION RESULTS FOR EACH MODEL AND COMPONENT.

Component	Gemma3	Mistral	Qwen2.5	Qwen3
Router segment accuracy	0.955	0.940	0.905	0.940
NLU slot micro F1	0.824	0.529	0.869	0.936
DM exact-match accuracy	0.996	0.996	0.940	1.000
NLG acceptance rate	0.941	0.941	0.912	1.000
Final average score	0.929	0.851	0.906	0.969

APPENDIX E
EXTRINSIC EVALUATION RESULTS

TABLE IV
EXTRINSIC EVALUATION RESULTS FROM 5 EXTERNAL TESTERS. SCORES ARE REPORTED ON A 1–5 LIKERT SCALE.

Evaluation category	Mean	Std. dev.
Request understanding	3.60	0.89
Task success and efficiency	4.00	0.50
Context tracking	3.20	1.10
Response naturalness	4.40	0.55
Robustness to unclear or invalid inputs	3.20	0.45
Overall score	3.68	0.27